

Blueberry

AI Dependency Validator

Project Guide

Team Members

Parthiban M – 24BYB0135

Nisha Rathi – 24BYB0137

Project Details

Overview

Blueberry is a VS Code extension that checks AI-generated package recommendations before a developer installs them. When tools like GitHub Copilot, Cursor, Claude Code, Gemini Code Assist, or Codeium suggest a third-party library inside a code completion, there is no guarantee that library actually exists, is maintained, or is safe. Blueberry sits between that suggestion and the terminal, verifying the package against its registry, checking the health of its source repository, and flagging anything that looks fabricated or deliberately deceptive.

Problem Statement

AI coding assistants generate package names the same way they generate any other token: by predicting what is statistically likely to follow, not by confirming that the package exists. This produces three recurring failure modes. First, hallucinated packages names that sound plausible but were never published to any registry. Second, typosquatted packages malicious libraries published under names one character removed from a legitimate, popular package, betting that a developer or an AI model will make that exact substitution. Third, low-reputation packages libraries that do exist but have no meaningful maintenance history, no source repository, or ownership patterns consistent with abandoned or hijacked projects.

Attackers have already started registering package names that LLMs are known to hallucinate, a technique referred to as slopsquatting. Because AI-assisted development treats these suggestions as trusted output and installation is usually one command away, the window between suggestion and compromise is short and the developer has little to go on beyond the model's confidence.

Project Scope

The project is deliberately narrow in its first phase: it supports Python packages sourced from PyPI only. This keeps the registry integration, the metadata schema, and the risk-scoring logic simple enough to build and validate within the project timeline, while still proving the core mechanism end to end. Support for npm, crates.io, Maven, and NuGet is scoped as future work rather than part of the initial build, since each registry has its own API shape, metadata conventions, and trust signals that would need separate handling.

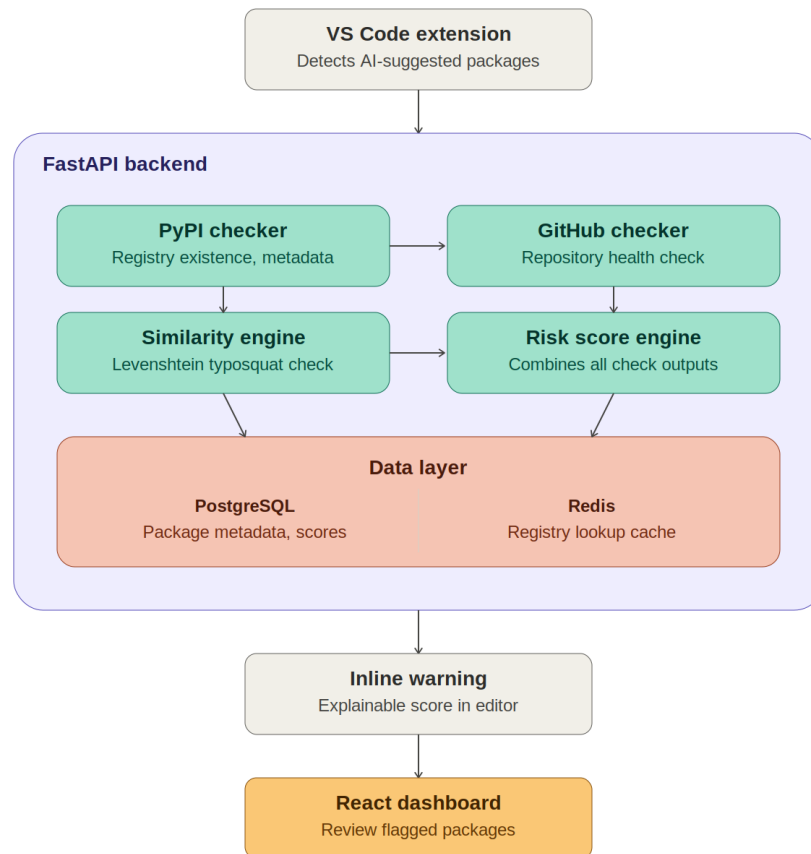
Core Features

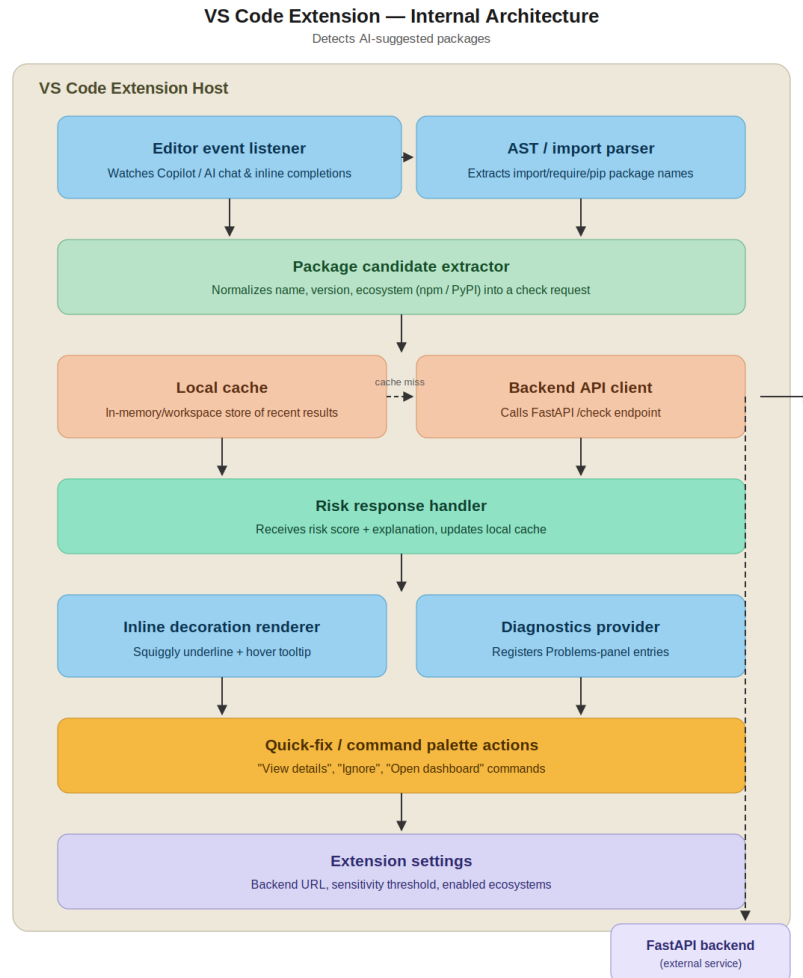
- VS Code extension that intercepts package names as they appear in AI-generated code or terminal commands
- PyPI registry verification to confirm a package actually exists and pull its published metadata
- GitHub repository verification to check whether the package's source repository is active, maintained, and consistent with its registry listing
- Levenshtein similarity detection to catch typosquatted names close to popular, high-traffic packages

- An explainable, rule-based risk score, so a developer sees why a package was flagged rather than a single opaque number
- A React dashboard for reviewing flagged packages and their scoring breakdown
- A FastAPI backend that coordinates the checks and serves both the extension and the dashboard

Architecture

The system is organized as a thin client and a central backend. The VS Code extension is the entry point: it detects package names as they surface in AI-generated suggestions and forwards them to the backend rather than performing any verification locally. The FastAPI backend runs four checks in sequence: a PyPI checker confirming registry existence and metadata, a GitHub checker validating the associated repository, a similarity engine comparing the name against known popular packages, and a risk score engine that combines the outputs of the previous three into a single explainable score. The result is returned to the extension as an inline warning, keeping the verification logic centralized and the extension itself lightweight.





Tech Stack

The backend is built on FastAPI for its async support and low overhead when making concurrent calls to external registries. PostgreSQL stores package metadata and historical scoring data; Redis caches registry lookups so repeated checks on common packages don't re-hit PyPI or GitHub on every request. The dashboard is a React and TypeScript single-page application. Docker is used to containerize the backend and its dependencies for consistent local development and deployment.

Development Timeline

The build is planned across six weeks. Week one covers the backend skeleton and PyPI API integration. Week two adds GitHub API verification and the risk engine. Week three is dedicated to the VS Code extension. Week four builds the React dashboard. Week five is integration and testing across the full pipeline. Week six covers documentation and the final demo.

Future Work

Beyond the current build, the natural extensions are multi-registry support (npm, crates.io, Maven, NuGet), ML-based hallucination detection to complement the current rule-based approach, CI/CD integration so checks run automatically in a build pipeline rather than only at suggestion time, and an enterprise dashboard for teams tracking dependency risk across multiple repositories.

Conclusion

Blueberry validates AI-generated dependency recommendations before installation, using lightweight and explainable techniques. It targets a narrow, well-defined problem Python packages via PyPI and builds the verification pipeline in a way that extends cleanly to other registries and detection methods once the core mechanism is proven.

Domain Knowledge

Software Supply Chain Security

Modern software is assembled, not written from scratch. A typical project pulls in dozens or hundreds of third-party packages, each of which can pull in its own dependencies. This creates a supply chain: the security of an application depends not just on its own code but on every package it imports, transitively. Supply chain attacks exploit this by compromising a dependency rather than the target application directly. An attacker who controls one widely-used package gains access to every project that installs it. This class of attack has grown steadily as package ecosystems like PyPI, npm, and crates.io have scaled into the millions of published packages, most maintained with minimal oversight.

Typosquatting and Dependency Confusion

Typosquatting in the package ecosystem context means publishing a malicious package under a name that closely resembles a popular, trusted one—for example, 'requeests' instead of 'requests'. The attack relies on a human or automated process making a small, plausible mistake when typing or selecting a package name. A related technique, dependency confusion, exploits the way some tooling resolves package names across public and private registries, causing an internal package name to accidentally pull a public, attacker-controlled package instead. Both techniques depend on the same underlying weakness: package managers install code by name, without any inherent guarantee that the name maps to the entity the developer actually intended.

AI-Generated Code and Package Hallucination

Large language models trained to autocomplete code will, when asked to suggest an import or a pip install command, sometimes generate a package name that sounds correct but does not exist. This happens because the model is predicting a statistically plausible token sequence rather than querying a registry. Security researchers have documented this pattern closely enough that it has its own name: slopsquatting, describing the practice of attackers pre-registering package names that models are known to hallucinate, so that when a developer trusts the AI's suggestion and runs the install command, they pull down a malicious package instead of getting an error. This is a distinct risk from traditional typosquatting because the victim isn't making a typing mistake; they're trusting an AI-generated suggestion at face value.

Package Registry Trust Signals

Registries like PyPI expose metadata that can be used to estimate how trustworthy a package is, even without deep code analysis. Publication date and version history indicate whether a package has an established track record or was created recently, which is common for both hallucination-bait packages and legitimate but risky new projects. Download counts and reverse-dependency counts indicate real-world adoption. Whether a package links to an active, matching source repository is another strong signal. A listing with no repository, or one pointing to an empty or unrelated GitHub project, is a red flag. None of these signals is conclusive on its own, which is why a workable trust assessment combines several of them rather than relying on any single check.

String Similarity for Typosquat Detection

Levenshtein distance measures how many single-character edits (insertions, deletions, or substitutions) are needed to turn one string into another. It is a standard tool for typosquat detection because it directly models the kind of mistake typosquatting exploits: a name that is one or two edits away from a popular package name is exactly the kind of substitution an attacker banks on. Applying this against a list of high-traffic package names lets a system flag suspicious near-matches without needing to understand anything about what the package actually does.

Explainability in Automated Risk Scoring

A risk score is only useful to a developer if they can see why it was assigned. Rule-based scoring (where each check (registry existence, repository health, name similarity) contributes a known, fixed weight to the final score) trades some of the flexibility of a machine-learned model for transparency. A developer who sees a warning triggered by 'no matching GitHub repository, 92% similarity to a popular package' can make an informed decision quickly. That transparency matters more in a security tool than in most other applications, since a warning nobody trusts or understands tends to get ignored.

Process Model

Model Selected: Agile (Iterative-Incremental Development)

Blueberry is being built using an Agile, iteration-based approach, structured as a sequence of short, functionally complete builds rather than a single upfront design followed by a long implementation phase.

Justification

The six-week development timeline is already organized as a sequence of working increments: backend and PyPI integration in week one, GitHub verification and the risk engine in week two, the VS Code extension in week three, the dashboard in week four, integration and testing in week five, and documentation in week six. Each week produces a component that can be tested on its own before the next one is layered on top, which is the defining characteristic of an iterative-incremental process rather than a waterfall one.

This fits the nature of the project for a few concrete reasons. The system has clearly separable modules the PyPI checker, the GitHub checker, the similarity engine, and the risk score engine are independent pieces of logic that can be built, tested, and refined one at a time before being wired into the full pipeline. Building them incrementally means a defect in the similarity engine, for instance, doesn't block progress on the extension or the dashboard, since those integrate with the backend through a stable API contract rather than depending on every internal component being finished simultaneously.

The risk scoring logic in particular benefits from iteration. Rule-based scoring is easy to get approximately right and hard to get exactly right on the first attempt weighting how much a missing GitHub repository should matter relative to a high similarity score is the kind of decision that improves once there's a working system to test against real package names. A waterfall approach, which would require finalizing the scoring rules during an upfront design phase before any code exists, offers no opportunity to adjust those weights based on how the checks actually behave once integrated.

Finally, a first release with a narrowly scoped feature set (PyPI only, four fixed checks, one client) doesn't carry the coordination overhead that would justify heavier process frameworks built for larger teams or longer timelines. A two-person team working a six-week build gets more value from short, working increments and continuous integration testing (week five is set aside for exactly this) than from formal sprint ceremonies or a rigid phase-gate structure. The process model matches the size of the team and the project.